

VoxelsFinal.py

This script does procedural voxel generation. It is used by calling functions and passing variables between them. It was specifically designed to be modular. I did consider implementing this as a node graph within blender, then decided that's not a very interesting problem to solve and the indifference of others is not worth my time. This also exist as a single script when really it should be a few. I find python in blender often have issues importing scripts and the easiest way to avoid that is to not import anything.

The workflow is relatively straight forward. A voxel array of dimensions x, y, z is created of type float. These can be created by one of the functions or in what ever other way. These can also be added, divided or have any other mathematical function applied to them. In its final form it is to be called **land_array**

Two more arrays are needed, one of type int which describes the contents of the voxels. Its will be used to map the texture which are tiles of soil or sand or moss or rock or what ever. The other is of type float and describes how smooth that part of the voxel object is. Best values are between -0.25 for rough and 0.45 for smooth. These two arrays can be generated which ever way, but is usually generated from the land_array using one of the relevant functions. **create_layered_array function** can generate all three arrays at once. In their final form these are called **contentsArray** and **smooth_value_array**

These three arrays are sufficient to generate voxels.

The main area of interest is the Call functions section in the code which is towards the bottom of the script.

```
#####
```

#Call functions

```
#####
```

This is the only area you need to modify to generate voxels. It contains a lot of blocked out code which is just examples of the function calls that's possible. The function themselves have more descriptions of what they do and what their input and outputs are. The non blocked out code includes.

```
random.seed(402) #make results reproducible
```

```
np.random.seed(4800)
```

The values can be anything and merely ensures that the voxels generated is the same every time the code runs.

```
if bpy.context.active_object and bpy.context.active_object.mode != 'OBJECT':
```

```
    bpy.ops.object.mode_set(mode='OBJECT')
```

Blender needs to be in object mode for this I think.

Define the dimensions of our grid

```
dims = (40, 55, 30) # (x, y, z)
```

Size of the voxel object generated, larger is slower though so I recommend using smaller values for testing and larger for the final where possible.

```
remapping = {
```

```
    1: -0.25,
```

```
    2: 0.5,
```

```
    3: 0.45,
```

```
    4: -0.25,
```

```

    5: 0.45,
}

```

```

remapped_smooth_value_array = remap_float_array_by_int_id(contentsArray,
smooth_value_array, remapping)

```

These are how smooth the different voxel types are. The default values generated are kinda random so this can be used to make them not random. Best values are between about -0.25 for very rough to about 0.45 for very smooth.

```

uvParams = {
    "width" : 6,
    "height": 14,
    "buffer": 0.0005
}

```

this is how the UV coordinates would be generated. Height is the amount of different kind of voxels that exist and width is the variations on the textures for each. The first is top, then side that meets top, then bottom, then side that's basically part of a flat wall, then side that meets floor and side that meets both top and floor. This can be extended with script to include anything really. If the dimensions given is smaller than the UV map generated the Uvs will spill out of the 0,1 space and repeat on the voxels. Buffer is used to avoid artifacts where tiles bleed into neighbors.

#x, y, z, 6, 2 index of uv cords array for each face on each voxel

```

face_array = np.zeros((dims[0], dims[1], dims[2], 6, 2), dtype=int) #-x, x, -y, y, -z, z
face_array = AssignContentsToFaces(contentsArray, face_array)

```

Face data is separate from contents for maximum flexibility. So faces can be remapped to anything. Contents is only used to generate the original face uv's and isn't really used for anything else. The thing to notice with this is faces exist for all voxels whether or not they are drawn, the same can be said of the smooth value array and the content array. So empty space in the voxel object does have content and smoothness and faces even though its empty. That makes math on these things lot easier.

#make int array from float array

```

final_binary_data = threshold_array(land_array, cutoff=0.3)

```

The array function itself would consider anything larger than 0 solid, this gives more control over that. Lower values should result in more solid voxels and higher values in more empty space. The original voxel generated is of type float and exist as a range of values and whats considered solid and not is arbitrary so it can be rendered which ever way.

```

def modify_face_UVS(uvFunctionParams, vert_coords, voxel_contents_array, face_array, x, y, z,
facepos):
    """

```

```

    uvFunctionParams: parramaters passed in
    vert_coords: last 4 coords of last face
    voxel_contents_array: the whole voxel array
    face_array: the whole voxal faces array
    x, y, z: position coordinates
    facepos: indicate direction of face or position in array
    """
    coord1 = Vector(vert_coords[len(vert_coords) - 1])

```

```

coord2 = Vector(vert_coords[len(vert_coords) - 2])
coord3 = Vector(vert_coords[len(vert_coords) - 3])
coord4 = Vector(vert_coords[len(vert_coords) - 4])
if facepos <= 3: #sides only, not top or bottom #-x, x, -y, y, -z, z
    face_array = check_and_update_face_uv(
        voxel_contents_array=voxel_contents_array,
        face_array=face_array,
        x=x, y=y, z=z,
        face_index=facepos,
        uv_set_above_only=[3, face_array[x][y][z][facepos][1]], #existing row 3 column
        uv_set_below_infront_only=[5, face_array[x][y][z][facepos][1]], #existing row 5 column
        uv_set_both=[4, face_array[x][y][z][facepos][1]] #existing row 4 column
    )

face_array = update_uv_if_facing_direction(
    v1=coord1, v2=coord2, v3=coord3, v4=coord4,
    direction=Vector((0.0, 0.0, -1.0)),
    cutoff=0.5,
    uv_array=face_array,
    x=x, y=y, z=z,
    facepos=facepos,
    uv_x=0, uv_y=face_array[x][y][z][facepos][1] #existing row 0 column
)

return face_array

```

```

uvFunction = modify_face_UVS
uvFunctionParams = {
    "Placeholder" : 6,
}

```

this is the function that will remap the faces. It is not called here it is just put in a variable. It is called from the function that creates the voxels when it creates the voxels. UvFunctionParams is not currently used but exist since it may be needed in the future. When the function is called it receives a whole bunch of data that may be useful for remapping faces. Currently it only calls two functions check_and_update_face_uv and update_uv_if_facing_direction. The first function changes the UV position of the side faces to better match their surroundings and the second would map anything that mostly point up as a top surface.

GenerateVoxels(final_binary_data, remapped_smooth_value_array, face_array, uvParams, uvFunction, uvFunctionParams)

This function call makes the voxel object. **remapped_smooth_value_array** can be replaced with **smooth_value_array** to use the original random smooth values.

This script is also an experiment in programming with AI in this case gemini. The voxel code is my original voxel code, most of the other functions is AI generated and then debugged and modified. That makes it very easy to expand this script to do all sorts of things. I find the AI work best if you very specific and debug as you go along. Complex things are best broken into parts and/or iterated upon. I did not have much success implementing complex things I don't understand very well myself. The AI

is also very good for time saving throw away code which I mostly used for making the image maps for the voxels.

This is a typical example of the type of prompt I use. “I’d like a function that takes in a tuple of 3 int values and creates a numpy array of dimensions x, y, z of type float using those three values. The function should also take a vector and a dictionary with value pairs of float float and a float named min and a float named max. These will be used to fill the array. The array will be filled with a gradient perpendicular to direction based on the values in the dictionary. The first value in the dictionary will indicate where and be in the range 0 to 1 and the second indicate the relative value of the voxels between min and max at that position and would also be in the range 0 to 1. so a pair of 0, 1 would indicate that at the start of direction the array would contain max value voxels, if that was to be followed by a pair 0.25 and 0.5 it would indicate a quarter of the way between the start of direction and its end the voxels would contain a value that’s half way between min and max. Values between those that exist in the dictionary should be interpolated from those that do exist. If there are no first values for 0 in the dictionary it should be assumed the pair is 0, 0 and if there are no values for 1 in the dictionary it should be assumed the pair is 1, 1. The array is to be returned “